

# ORGNZ-07: Advanced Permission Patterns

Series: ORGNZ | Notebook: 7 of 9 | Created: January 2026 | Last Updated: 01/28/2026

## Overview

This notebook covers advanced permission patterns including record-level permissions, field-based access, and combining multiple access control mechanisms for enterprise-scale data governance.

## Prerequisites

| Requirement | Details |  
|---

## Table of Contents

- 1. [Record-Level Permissions](#)
- 2. [Record-Level Policy Examples](#)
- 3. [Field-Level Access](#)
- 4. [Combined Permission Patterns](#)
- 5. [Enterprise Architecture Patterns](#)
- 6. [Testing Permissions](#)
- 7. [Best Practices](#)

-----|-----|

| **Dynatrace Account** | Account-level administrative access |

| **Permissions** | IAM policy management permissions |

| **Knowledge** | Completed ORGNZ-04 through ORGNZ-06 |

## Learning Objectives

---

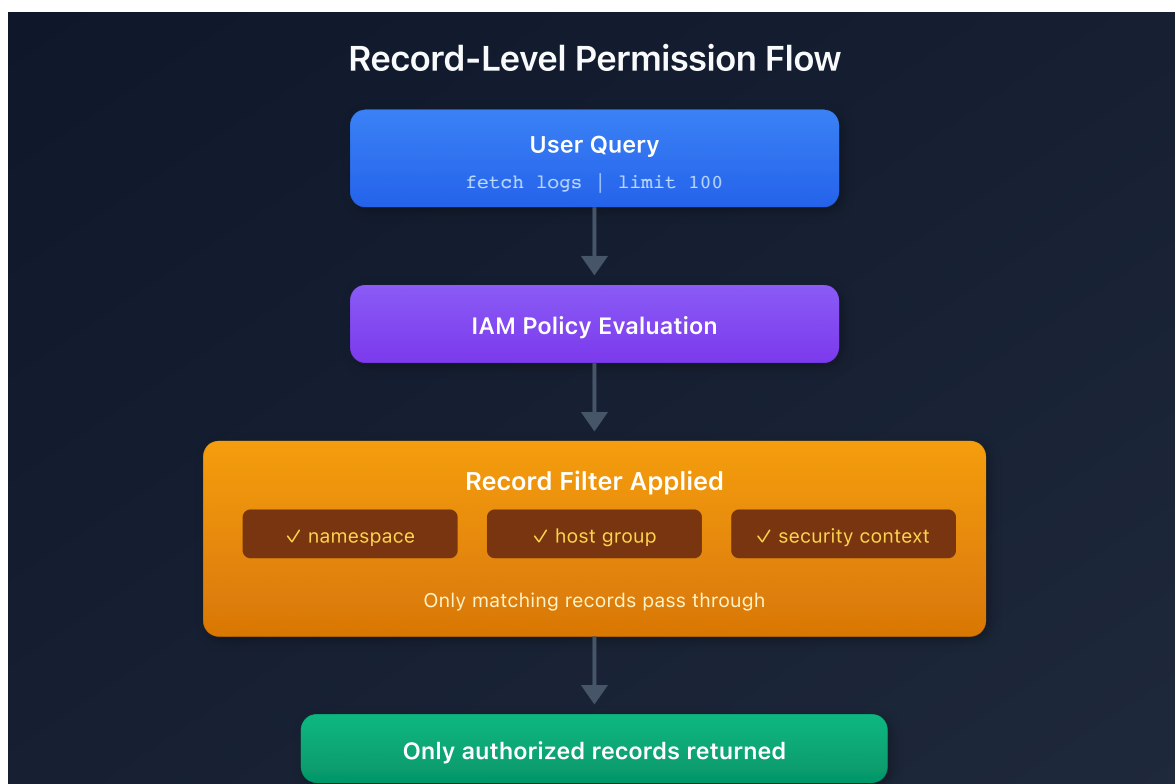
By the end of this notebook, you will:

- Implement record-level permissions using various attributes
- Understand field-level access restrictions
- Combine bucket, record, and field-level policies
- Design enterprise-scale permission architectures

## Record-Level Permissions

---

Record-level permissions filter data at query time based on record attributes:



## Supported Record-Level Conditions

| Condition                                | Description          | Example                         |
|------------------------------------------|----------------------|---------------------------------|
| <code>storage:k8s.namespace.name</code>  | Kubernetes namespace | <code>= 'production'</code>     |
| <code>storage:k8s.cluster.name</code>    | Kubernetes cluster   | <code>= 'main-cluster'</code>   |
| <code>storage:host.name</code>           | Host name            | <code>= 'web-server-01'</code>  |
| <code>storage:dt.host_group.id</code>    | Host group           | <code>STARTSWITH 'prod-'</code> |
| <code>storage:service.name</code>        | Service name         | <code>= 'checkout'</code>       |
| <code>storage:aws.account.id</code>      | AWS account          | <code>= '123456789012'</code>   |
| <code>storage:gcp.project.id</code>      | GCP project          | <code>= 'my-project'</code>     |
| <code>storage:dt.security_context</code> | Custom context       | <code>MATCH ('team-*')</code>   |

## Record-Level Policy Examples

### Kubernetes Namespace Isolation

```
{
  "name": "production-namespace-access",
  "description": "Access only to production namespace data",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH 'default_'; ALLOW storage:logs:read WHERE storage:k8s.namespace.name = 'production';"
}
```

### Host Group Based Access

```
{
  "name": "web-tier-access",
  "description": "Access to web tier hosts only",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH 'default_'; ALLOW storage:logs:read, storage:metrics:read WHERE storage:dt.host_group.id STARTSWITH 'web-';"
}
```

## Combined Namespace and Host Group

```
{
  "name": "prod-web-tier-access",
  "description": "Production web tier only",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read WHERE
storage:k8s.namespace.name = 'production' AND storage:dt.host_group.id
STARTSWITH 'web-';"
}
```

## Cloud Account Isolation

```
{
  "name": "aws-team-account-access",
  "description": "Access to specific AWS account",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read, storage:metrics:read,
storage:spans:read WHERE storage:aws.account.id = '123456789012';"
}
```

## Field-Level Access

Control access to specific fields within records:

| Use Case                | Implementation                  |
|-------------------------|---------------------------------|
| Hide sensitive fields   | Deny access to specific fields  |
| Mask PII                | Field-level policies            |
| Compliance requirements | Restrict access to audit fields |

## Field-Level Policy Example

```
{
  "name": "restricted-field-access",
  "description": "Hide sensitive fields from general users",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read; DENY
storage:logs:read:user.email, storage:logs:read:user.ip_address;"
}
```

## Combined Permission Patterns

---

### Pattern 1: Layered Access Control

Combine bucket and record-level permissions:

```
Layer 1: Bucket access
  ALLOW storage:buckets:read WHERE bucket-name IN ('team_logs',
'shared_logs')

Layer 2: Record filtering
  ALLOW storage:logs:read WHERE k8s.namespace.name = 'team-namespace'

Layer 3: Field masking (optional)
  DENY storage:logs:read:sensitive_field
```

### Pattern 2: Multi-Team Shared Bucket

Multiple teams share a bucket with security context isolation:

```
// Team A policy
{
  "name": "team-a-shared-bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name = 'shared_logs'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ('team-a*');"
}

// Team B policy
{
  "name": "team-b-shared-bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name = 'shared_logs'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ('team-b*');"
}
```

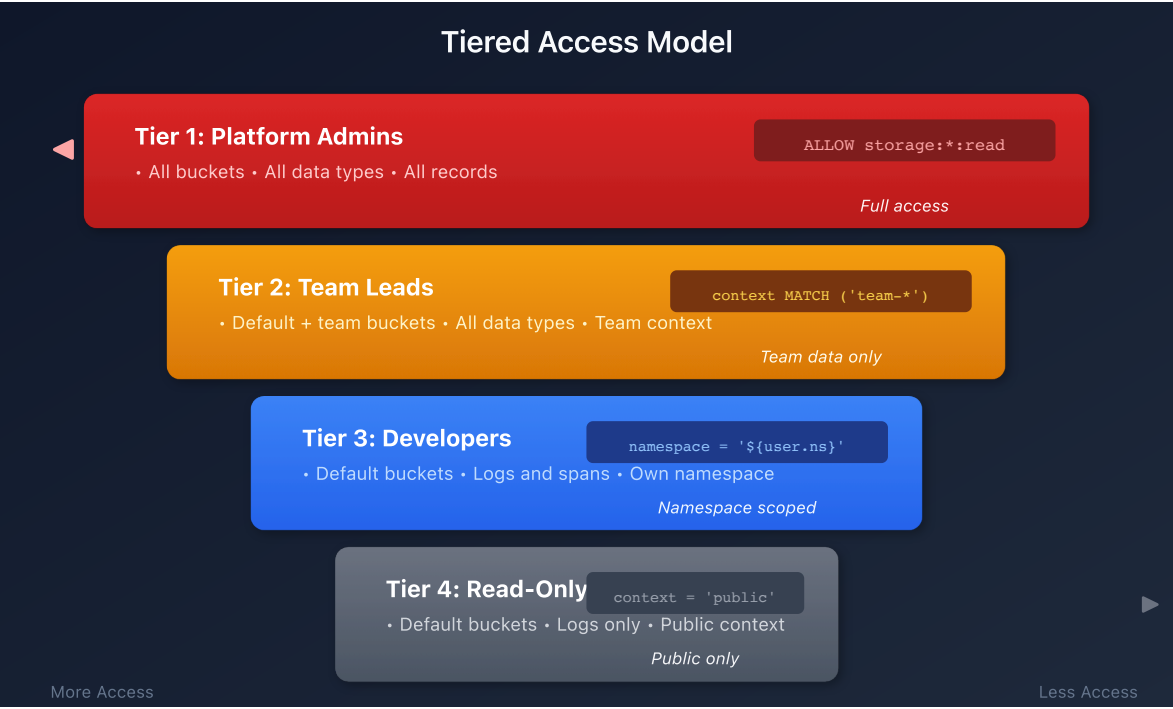
### Pattern 3: Environment-Based Tiering

```
// Production access (restricted)
{
  "name": "production-access",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH 'prod_'; ALLOW storage:logs:read WHERE storage:dt.security_context = 'env:production';"
}

// Non-production access (broader)
{
  "name": "non-production-access",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH 'default_'; ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ('env:dev*', 'env:staging*', 'env:qa*');"
}
```

# Enterprise Architecture Patterns

## Tiered Access Model



## Geographic/Regulatory Model

| Region | Buckets | Context   | Policy                                                                 |
|--------|---------|-----------|------------------------------------------------------------------------|
| EU     | eu_*    | region:eu | bucket-name STARTSWITH 'eu_' AND security_context MATCH ('region:eu*') |
| US     | us_*    | region:us | bucket-name STARTSWITH 'us_' AND security_context MATCH ('region:us*') |

## Testing Permissions

```
// Test namespace-based access
fetch logs
| filter isNotNull(k8s.namespace.name)
| summarize count = count(), by:{k8s.namespace.name}
| sort count desc
```

```
// Test host group access
fetch logs
| filter isNotNull(dt.host_group.id)
| summarize count = count(), by:{dt.host_group.id}
| sort count desc
```

```
// Verify accessible data distribution
fetch logs
| summarize
    total = count(),
    withSecurityContext = countIf(isNotNull(dt.security_context)),
    withNamespace = countIf(isNotNull(k8s.namespace.name)),
    withHostGroup = countIf(isNotNull(dt.host_group.id))
```

## Best Practices

---

| Practice                        | Rationale                            |
|---------------------------------|--------------------------------------|
| Start with least privilege      | Grant minimum required access        |
| Use groups, not individuals     | Easier management, consistent access |
| Document all policies           | Audit trail and governance           |
| Test policies before deployment | Prevent access issues                |
| Regular access reviews          | Remove unnecessary permissions       |
| Use MATCH for array fields      | Required for correct evaluation      |
| Combine bucket + record level   | Defense in depth                     |

## Next Steps

---

Continue with the ORGNZ series:

- **ORGNZ-08**: Grail Segments



# References

---

- [Configure advanced permissions with security context](#)
  - [IAM policy reference](#)
  - [Enhance data management with Grail](#)
- 

*This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*